

The Pattern Mirror

A longitudinal bias-pattern analysis tool for managers

Surfaces bias patterns in a manager's own hiring writing, with verified evidence, so they can decide what to change.

An applied-AI and full-stack engineering project

Built independently across UBS Tomorrow's Talent Programme 2026



Bernice Koh

PROGRAMME

UBS Tomorrow's Talent Programme 2026 · Technology Track

BACKGROUND

NTU · Double Degree in Business and Computer Science

Built with guidance from **Marcus Ng** (Tech Buddy) and **Seema Sanghavi** (Tech Mentor).

What's in this deck

The product

- 03 Why it exists
- 04 What it does
- 05 Writing tools for managers
- 06 Seeing your own patterns
- 07 The HR view

The engineering

- 08 Inside the bias-detection engine
- 09 LLM as a Judge
- 10 Research behind the Judge
- 11 How the dictionary grows itself
- 12 Architecture & tech stack
- 13 The data model

Wrap-up

- 14 What's next
- 15 What I learned
- 17 Repo & contact

Where bias training stops, this begins.

01

The gap

Managers complete fair-hiring and inclusivity training, but nothing checks whether it changes the job descriptions, interview feedback, and promotion writeups they actually produce. The training is theoretical; the writing is where hiring decisions are made.

02

The insight

Bias in that writing is a *pattern*. Any one document reads as unremarkable; the tendency only shows across the whole body of writing over time — which no one reviews.

03

The response

The Pattern Mirror flags bias-coded language in each document as it is written, checks each writeup against the reference it should match — a JD's stated criteria, an employee's peer feedback — and surfaces the patterns that recur across documents over time.

Evidence in, decision theirs

A flag — a specific span of a manager's writing the tool marks as potentially biased, shown with a plain reason and a citation. The manager decides what to do with it.

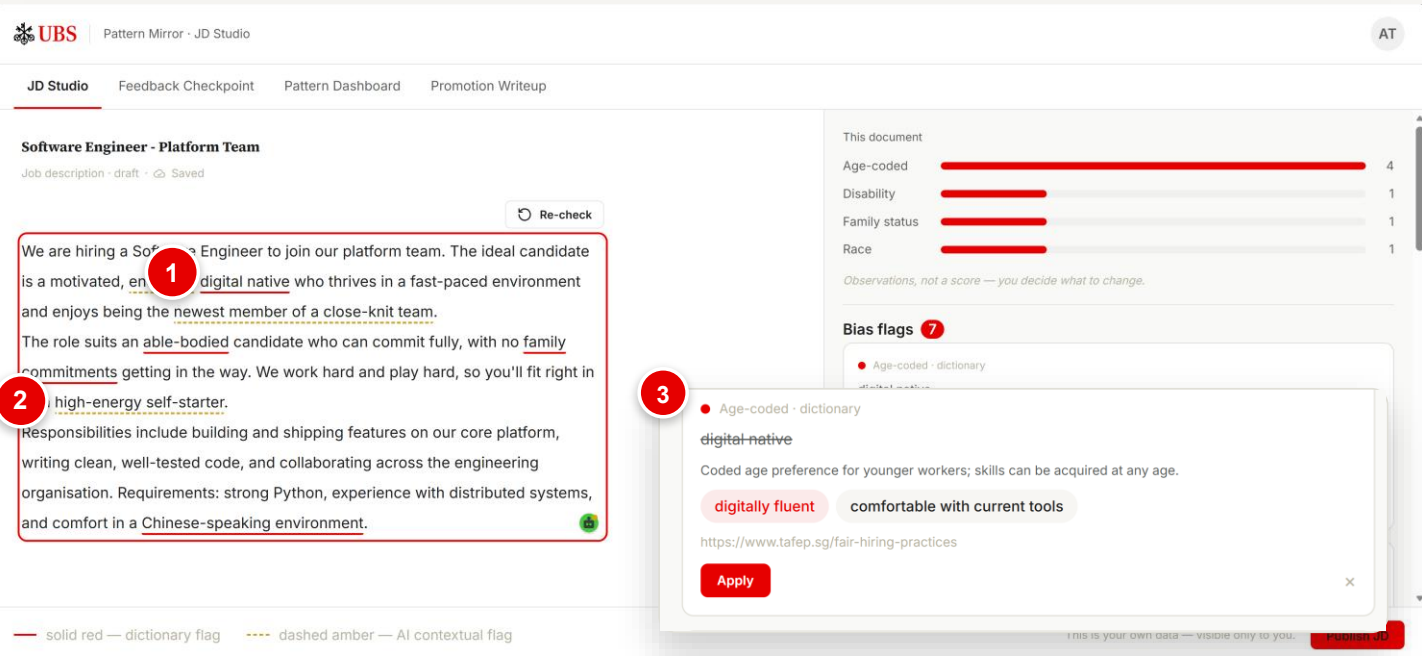
What it does	
Every flag is cited	Each flag carries a curated, research- or regulation-backed citation (e.g. TAFEP, Singapore's fair-employment authority). A flag that cannot be backed by a source is dropped, never shown uncited.
Patterns are statistically significant	A recurring tendency is only called a pattern once it passes a statistical test. A single flag is never a pattern.
HR sees aggregates only	HR gets firm-level trends. It cannot open or read any individual manager's writing.

Design invariants	
Mirror, not judge	Shows patterns; never scores or penalises the manager.
Private by architecture	HR cannot read any individual manager's writing; the data has no path there.
Non-blocking	Every flag is dismissible; submission is never prevented.
Always cited	Peer-reviewed research or regulatory guidance behind every flag.

Not a chatbot · **Not** a training module · **Not** a hiring gate — the manager keeps decision authority throughout.

Three writing tools, one bias-detection pipeline underneath

All three tools run the same bias-detection pipeline; the two drift-checking tools add a drift check – a comparison against a reference the writing should match (the JD's criteria, or peer feedback).



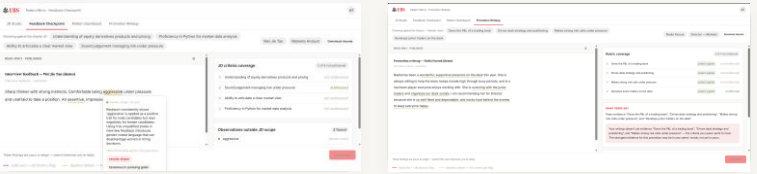
1 Instant flags on known phrasing
Established biased terms, caught the moment they're typed — no AI call.

2 A deeper check when you pause
An AI pass reads context and catches coded language a word list can't.

3 Evidence and fix in every flag
The phrase, a plain reason, rewrite chips and a citation. Apply resolves it.

TOOL	WHAT IT CHECKS
JD Studio	bias in a job description as it is written
Feedback Checkpoint	bias in interview feedback, and whether it covers the original JD's criteria
Promotion Writeup	bias in a promotion case, and whether it matches the employee's peer feedback

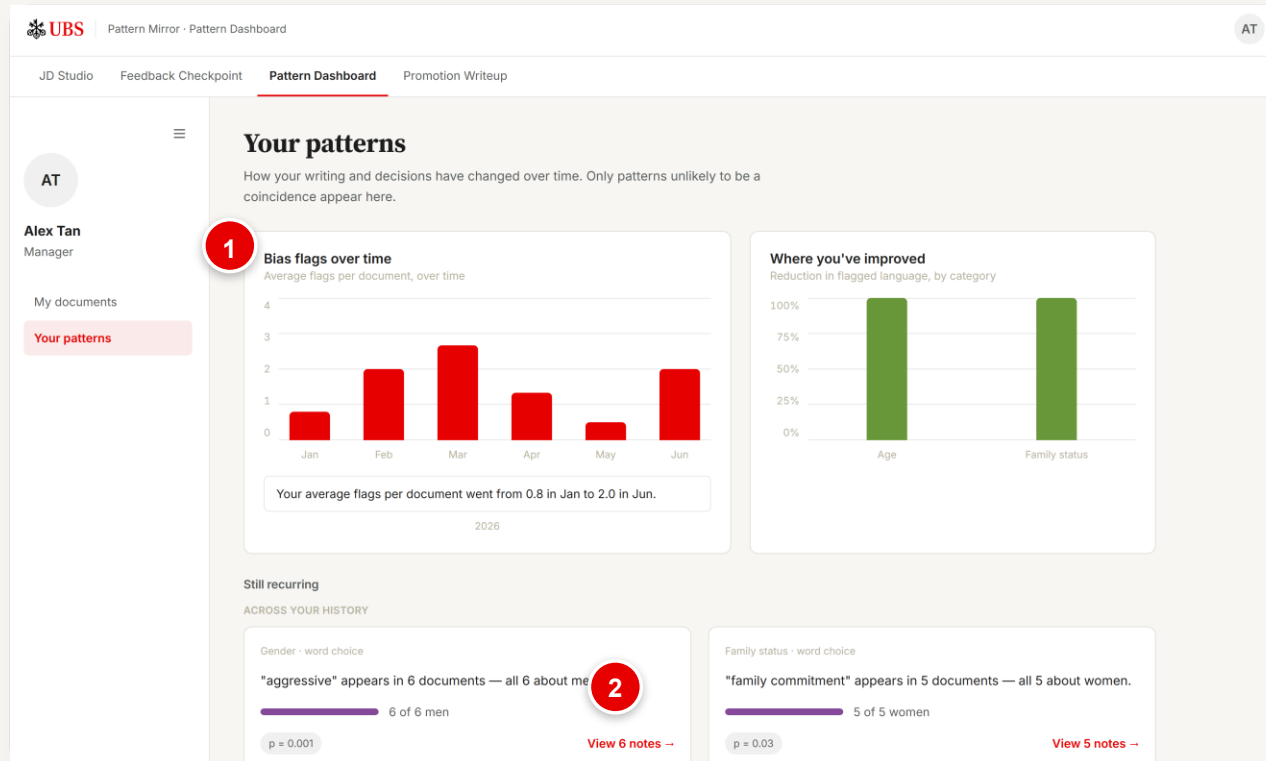
TWO MORE SURFACES · SAME PIPELINE



Feedback Checkpoint
reference · the JD's criteria

Promotion Writeup
reference · peer feedback

A pattern is a statistical finding, not an impression



1 Longitudinal view

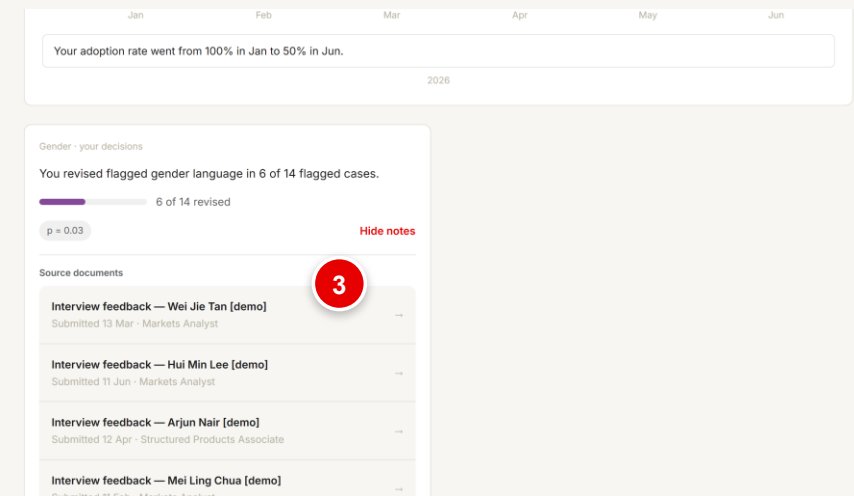
Flags per document over time, and where you improved — by TAFEP-protected characteristic: age, gender, race, nationality, religion, disability, family status.

2 Only significant patterns appear

Fisher's exact test keeps only patterns unlikely to be chance — 5 flags in 6 writeups counts; 5 across 100 documents doesn't. No AI here; it reads your stored flag history.

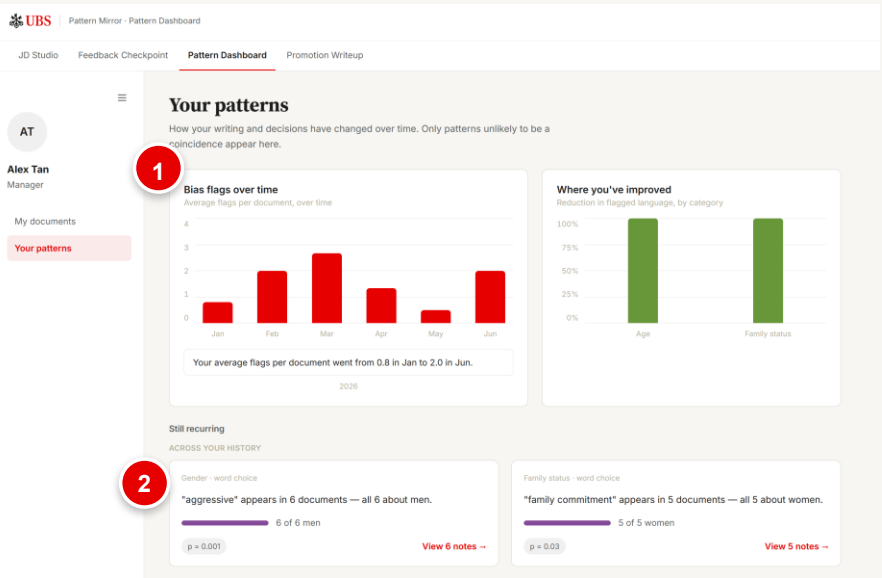
3 Drill into the evidence

Open a pattern for its source documents and the flags.



HR sees firm-level trends and cannot see any manager's writing

HR PORTAL · IS IT WORKING?



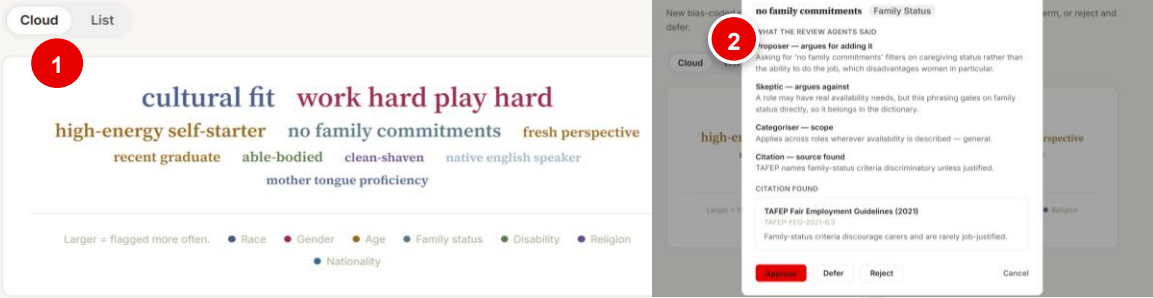
- 1 Aggregate only**
KPI tiles and firm charts — the footer states it plainly: enforced by the data model, not by policy. HR responses have no field for individual writing.
- 2 Small-cell suppression**
Any aggregate covering fewer than three managers is suppressed, so no figure can re-identify anyone — enforced in code (`hr_min_cell_size = 3`).

DICTIONARY REVIEW · WORD CLOUD

— HR Portal

Dictionary review queue

New bias-coded phrases the review agents flagged. Approve to add each as a dictionary term, or reject and defer.

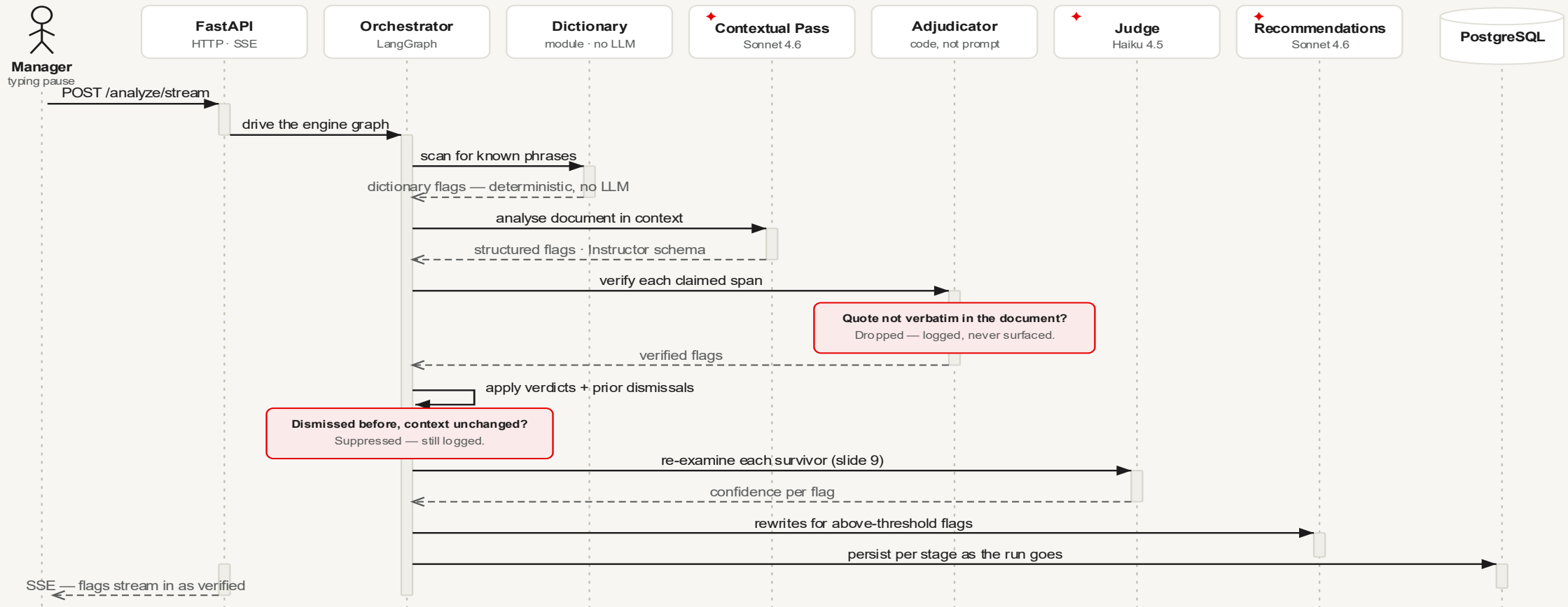


- 1 What the engine keeps proposing**
Candidate phrases awaiting review. Colour = protected characteristic; word size = queue ranking (larger = flagged more often).
- 2 Human decides**
Each candidate opens with the review agents' reasoning and its citation; HR approves, rejects, or defers.

The bias-detection engine

Five stages — every flag is verified against the document before it can reach the manager.

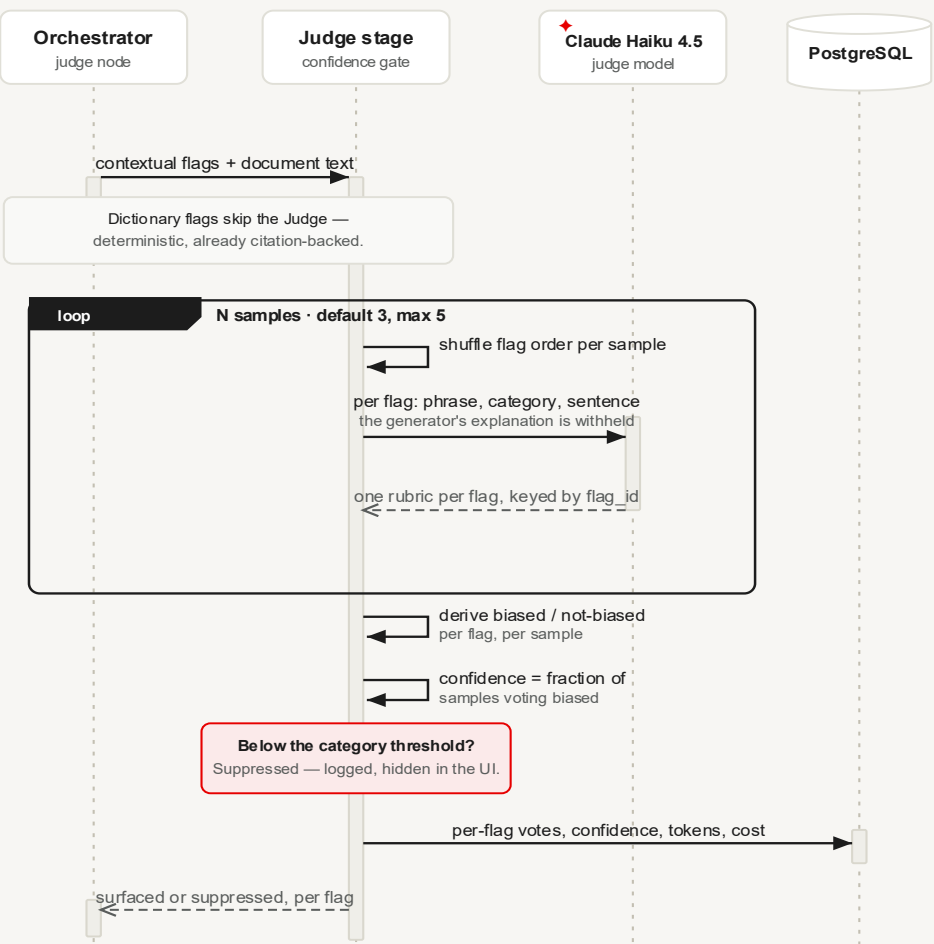
→ call ---> return ♦ LLM agent □ deterministic module ○ datastore □ dropped / suppressed — still logged



LangGraph orchestration · Instructor schema enforcement at every LLM boundary · logs every dropped or suppressed flag with its reason · degrades to dictionary-only without an API key — **never fails closed**.

A second model must independently agree, or the flag is held back

The Judge is a confidence gate — a flag is surfaced only if a different model, checking the document itself several times, consistently derives that it is biased. Low-agreement flags are suppressed, not shown.



A different model judges

Haiku judges what Sonnet flagged, with the generator's explanation withheld — it forms its own view from the document, not the argument for the flag. A model favours its own output; a different one avoids that.

A rubric, not a score

The Judge answers narrow boolean questions; the verdict is derived in code. A small model answers “is this a measurable capability?” far more reliably than “rate 0–1”.

Confidence is a frequency

N independent samples; confidence = the fraction that derive “biased”. A measured agreement rate, not a self-reported feeling.

Flag order shuffled per sample

Rubrics matched back by id, never by position — removing list-position bias from the verdict.

THE RUBRIC & THE RULE

Per flag, the Judge answers three booleans — `references_characteristic` · `gdor_plausible` · `stated_objectively` — and the verdict is derived in code:

```
biased ⇔ references_characteristic AND NOT (gdor_plausible AND stated_objectively)
```

GDOR — Genuine and Determining Occupational Requirement — TAFEP's narrow lawful exception: a characteristic may be referenced only if the job genuinely cannot function without it, and it is stated as a measurable capability (“must lift 15 kg”), not an identity trait (“young and energetic”).

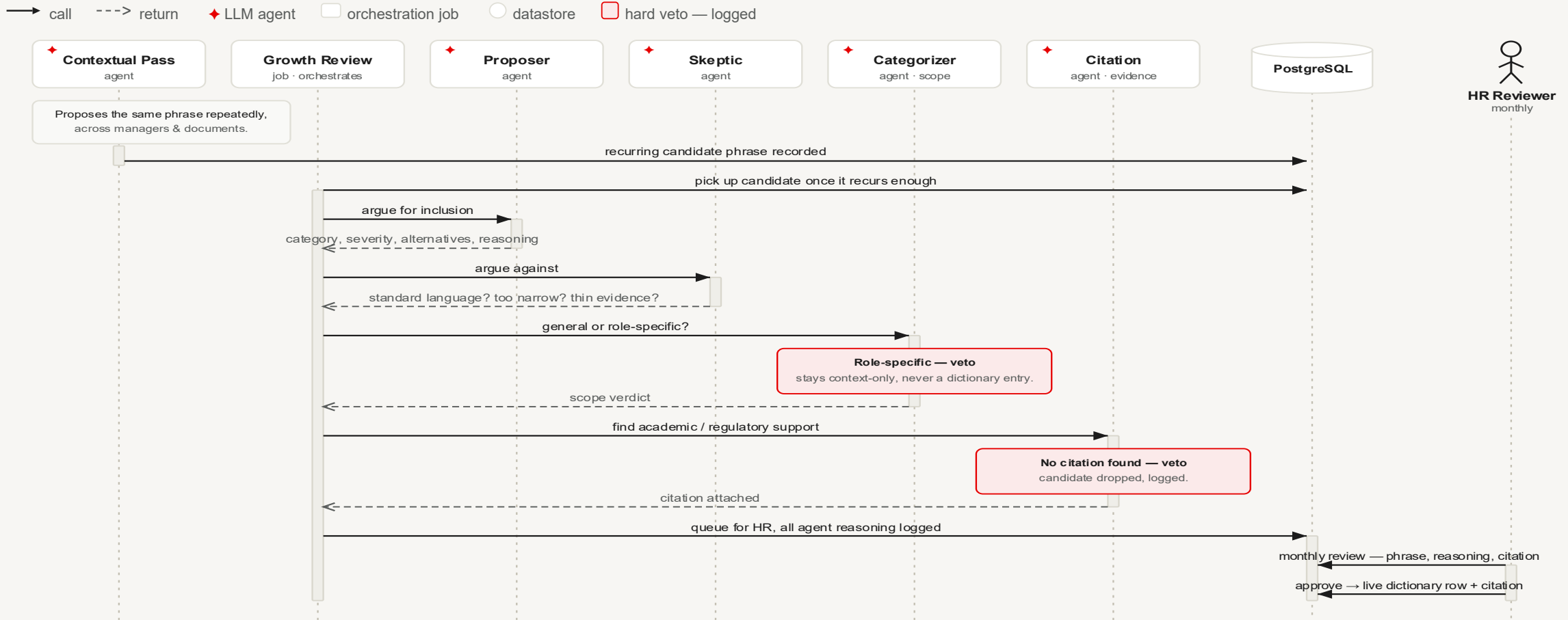
Every suppression is explainable after the fact — which rubric criterion failed, and how consistently, on the record per flag.

The Judge design follows the published evidence

DESIGN CHOICE	RESEARCH GROUNDING
A different model judges	Self-preference bias is causal — models favour output they recognise as their own, so grading is done with a different model than the one that generated it. Panickssery, Bowman & Feng 2024 · arXiv:2404.13076
A rubric, not a score	Checklist decomposition beats single-scalar judging on agreement and variance; scalar Likert judgments are unstable and lenient. CheckEval — Lee et al., EMNLP 2025 · arXiv:2403.18771 · Gu et al. survey · arXiv:2411.15594
Confidence is a frequency	Confidence comes from self-consistency across samples — black-box sampling is nearly as good as logprob access. Wang et al., ICLR 2023 · arXiv:2203.11171 · Xiong et al., ICLR 2024 · arXiv:2306.13063
Flag order shuffled	Judges favour items by list position; reordering can flip rankings, so flag order is shuffled before judging. Wang et al. 2023 · arXiv:2305.17926 · Zheng et al., NeurIPS 2023 · arXiv:2306.05685 · Bias in the Loop — Zhao, Esmaeili & Fard 2026 · arXiv:2604.16790
Calibration measured, not assumed	A hand-labelled gold set measures calibration error (ECE, Brier) against the real engine; the calibration map stays identity until the numbers justify fitting one. Tian et al., EMNLP 2023 · arXiv:2305.14975 · Anthropic, “Adding Error Bars to Evals” · arXiv:2411.00640 · Towards Provably Unbiased LLM Judges — Feuer, Rosenblatt & Elachqar 2026 · arXiv:2603.05485

The dictionary grows itself — agents propose, HR decides

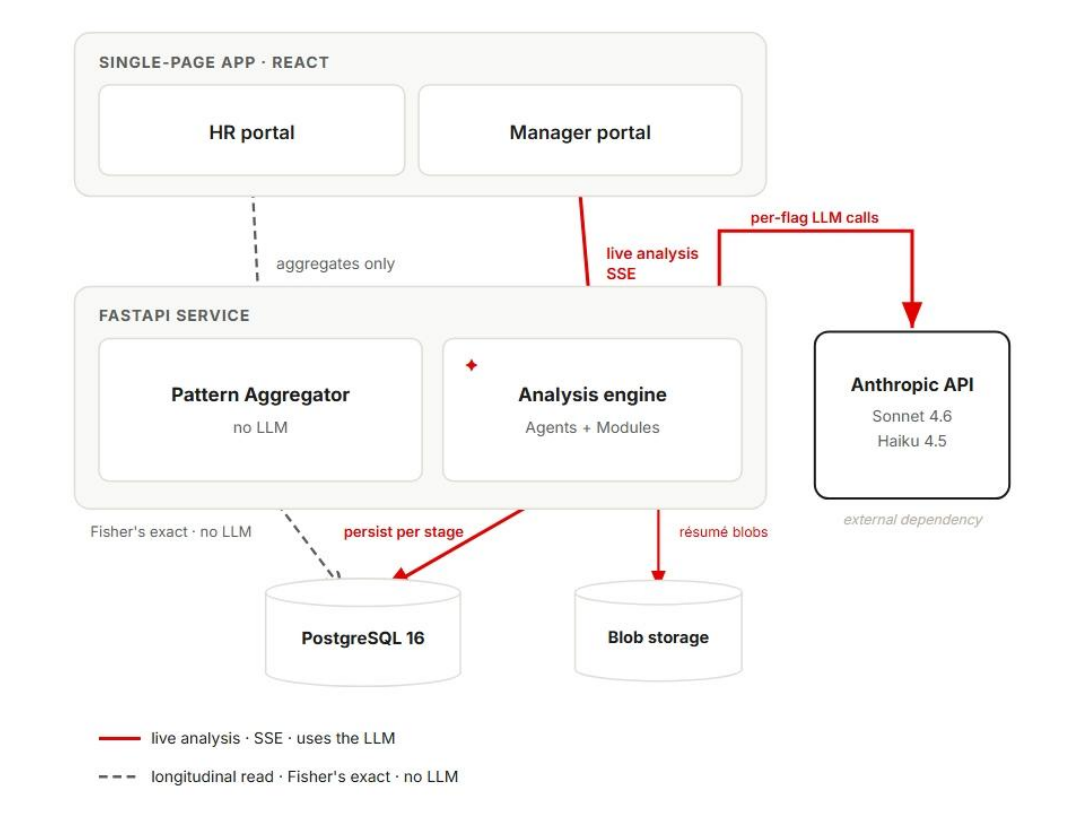
A phrase the engine keeps flagging across documents becomes a candidate; four agents review it independently; nothing enters the dictionary without a citation, a general scope, and a human approval.



Candidates and their examples are drawn from the same fair-employment corpus the engine is calibrated on — a proposed phrase is judged against real **TAFEP-labelled examples**, not a model's opinion in a vacuum.

One service, one SPA, one database

SYSTEM DIAGRAM

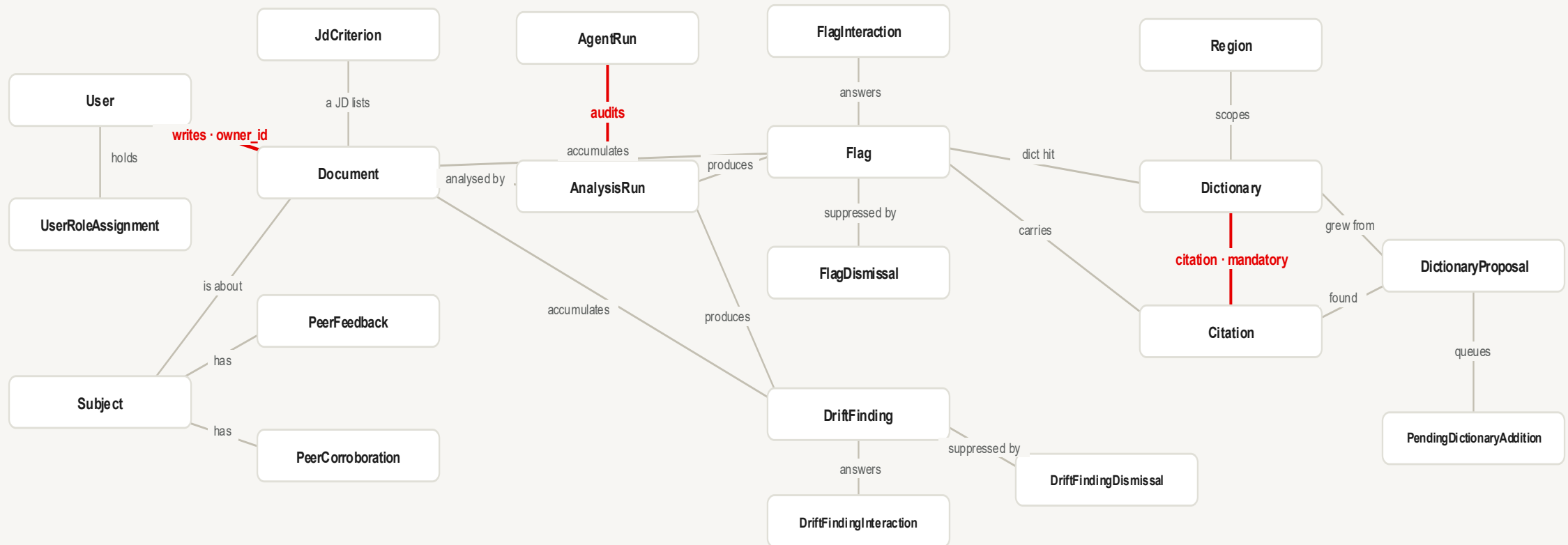


Two paths on one system — **live analysis** (SSE, uses the LLM) · **dashboard read** (no LLM).

LAYER	STACK
Frontend	React 19 + TypeScript · Vite · Tailwind · TipTap
Backend	Python 3.12 · FastAPI · SSE streaming
NLP	spaCy (lemma matching)
LLM	Anthropic: Claude Sonnet 4.6 (analysis, recommendations) · Haiku 4.5 (judge)
Orchestration	LangGraph · Instructor (schema-enforced outputs)
Database	PostgreSQL 16 · SQLAlchemy 2 · Alembic · Blob Storage (local disk → Azure Blob in prod)
Statistics	scipy — Fisher's exact test
Quality gates	GitHub Actions CI · SonarQube Cloud · pytest + vitest
Deploy	Docker Compose → Azure Container Apps

Privacy and audit live in the schema

Entities and their relationships — the three **red edges** carry the privacy and audit guarantees.



User → Document · owner_id

Every manager query filters on `owner_id` ; HR never reads below the firm-wide aggregate. This one column is the privacy boundary.

Dictionary → Citation · mandatory

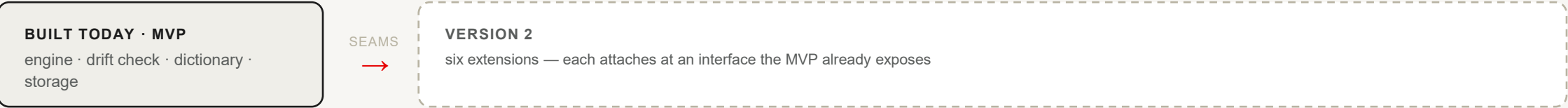
The foreign key is non-nullable — a biased-term entry with no evidence cannot exist.

AnalysisRun → AgentRun

Every LLM call is recorded with its model, tokens, and cost — the audit trail is a data-model fact.



Version 2 — each extension has its seam already built



EXTENSION	WHAT IT ADDS	THE SEAM ALREADY BUILT
Multi-model gateway	Route each agent to any provider at runtime, chosen by cost and agreement	Every model already speaks through one schema-enforced client (StructuredCompletionClient) — the gateway is a second implementation
Video as a drift reference (Gemini) Flagship next	Compare the interview recording against what the manager wrote; brings in Gemini for native video input	The drift check already swaps its reference corpus; DriftFinding.reference_kind is an enum — a video reference is a new value, not a new engine
RAG over historical writing	Reason over a manager's past documents in the moment	Documents are already stored (Postgres + blob); add pgvector , owner-scoped like every query
Hiring leaderboard	Rank a JD's candidates with a “why this résumé fits” summary	Composes existing résumés, JD criteria and the engine; a Redis cache holds the ranking
Region dictionaries	UK / US / EU / MY / ID / TH / PH / VN	The dictionary is region-scoped data — new rows plus citations, no engine change
Real feedback integration	Swap synthetic peer feedback for a system of record via ATS/HRIS connectors	The drift check consumes a corpus, indifferent to whether it was seeded or fetched

Nothing here needs a rewrite — the MVP left a clean edge for each.

Four weeks of building like it is production

Engineering practice

- Branch-and-PR git flow, Conventional Commits, squash-merge
- Issues as the unit of work
- CI on GitHub Actions — ruff, mypy --strict, eslint, pytest, vitest
- SonarQube gating merges
- Every architectural choice written up with its rationale at decision time
- Pre-commit hooks

AI engineering

- Agent orchestration with LangGraph
- Schema-enforced LLM output with Instructor
- LLM-as-a-Judge design — independence, rubric decomposition, self-consistency, calibration
- Prompt changes treated as a quality-regression surface
- Cost and latency budgeted per agent call

Domain

- How AI lands in a regulated setting: privacy by architecture, audit trails as product features, human-in-the-loop review, non-blocking assistance
- Fair-hiring regulation (TAFEP and SEA equivalents)
- Why evidence and dismissibility decide whether a manager trusts the tool

WHAT I'D DO DIFFERENTLY

Adopt **spec-driven development** from the start — writing the behaviour spec before the code, which I discovered partway through. It would have accelerated delivery and kept scope from creeping across the four weeks.

See the code, or get in touch



The repository

`github.com/Bernice-Koh/pattern-mirror`



LinkedIn

`linkedin.com/in/bernice-koh-sg`